

Final System Design Reference

AI-Based Phishing Resilience Recommendation System

Merged Document — Best of Both Versions

[Single-Session](#) | [Hidden Timer](#) | [Python Calculates](#), [GPT Writes](#) | [CSV Storage](#)

Project	Graduation Project — Azrieli College of Engineering
Model	Single session (~22 min), no database, no push notifications
Core Rule	Python computes all scores/risks/gaps. GPT only writes recommendations.
Version	3.0 — Final Merged (May 2026)

Table of Contents

1. System Objective & Design Decisions
 2. Core Constructs: AW, DB, SV
 3. Questionnaire Design & Scoring (9 items)
 4. Phishing Simulation Design (6 messages)
 5. Hidden Timer Logic & Penalty
 6. Risk & Gap Calculation (Python formulas)
 7. Error Type → Recommendation Content
 8. Gap Type → Recommendation Style
 9. GPT API Role & Prompt Template
 10. User Flow — Single Session
 11. Data Storage Schema (CSV)
 12. File Structure & Deployment
 13. Success Metrics
 14. Testing Scenarios
 15. Defense Explanation
 16. Future Work
- Appendix A: Conflict Resolution Log

1. System Objective & Design Decisions

The system helps students improve their resilience against phishing by comparing what they **report** about themselves (questionnaire) with what they **actually do** in a phishing simulation. It then generates a personalized recommendation.

Core principle: the error type determines WHAT to teach; the gap type determines HOW to teach.

The midterm research found that self-report explains only **2.5%** of variance in actual performance (R-squared = 0.025). This validates the need for simulation-based measurement.

Final MVP Decisions

Component	Decision	Rationale
Questionnaire	9 items (3 per dimension), Likert 1-6	Minimal viable measurement; 20 items was unnecessary for MVP
Simulation	6 messages (2 per dimension) from bank of 18	Covers all 3 dimensions; bank enables non-repeating Sim 2
Timer	Hidden automatic timer per message	More accurate than self-report; no disruption to flow
Post-message questions	Removed from MVP	Timer replaces time question; behavior question breaks realism
Calculation	Python computes all scores, risks, gaps, gap types	Deterministic & reproducible; AI outputs vary per call
AI role	GPT writes recommendation text ONLY	Receives pre-computed data; does not calculate anything
GPT output	Markdown report	Displayed directly via st.markdown(); no parsing needed
Storage	CSV file (upgradable to Sheets/Supabase)	Need data for final report; session_state loses data
Notifications	None (removed)	Push requires paid service; replaced by welcome screen
Session model	Single session (~22 min)	100% completion rate; no database needed
UI framework	Streamlit (Python)	Single language; free hosting; fast prototyping

2. Core Constructs: AW, DB, SV

Code	Construct	Meaning	What System Checks
AW	Security Awareness Deficiency	Difficulty recognizing phishing indicators (fake domains, senders)	Can the student distinguish fake institutional messages from legitimate ones?
DB	Risky Digital Behavior	Unsafe actual behavior (clicking fast, entering data without checking)	Does the student behave carefully in quiet/non-emotional phishing?
SV	Emotional/Situational Vulnerability	Susceptibility to urgency, fear, authority, curiosity, or financial temptation	Does pressure, threat, or reward affect the student's decision?

Important rule: In the questionnaire, all items are coded so that a **higher score = higher problem/risk**. This avoids confusion during scoring.

3. Questionnaire Design & Scoring

9 items total, 3 per dimension. Likert scale 1-6 (1 = strongly disagree / no problem; 6 = strongly agree / high problem). Items are written so that agreement = higher risk.

Dim	Item
AW	I find it difficult to distinguish a real domain from a fake domain.
AW	I sometimes rely on the visual design of the message instead of checking the sender.
AW	I am not always sure whether an institutional email is real or fake.
DB	I sometimes click links before fully checking them.
DB	I sometimes open attachments or links quickly.
DB	I sometimes enter personal details before verifying the official source.
SV	Urgent messages can make me respond too quickly.
SV	Threats such as account closure can pressure me.
SV	Unexpected rewards or financial offers can attract my attention.

Scoring Formulas

$AW_score = average(AW1, AW2, AW3)$

$DB_score = average(DB1, DB2, DB3)$

$SV_score = average(SV1, SV2, SV3)$

Convert to 0-1 self-report risk:

Self_Report_Risk = (score - 1) / 5

Example: score = 2.0 → Self_Report_Risk = (2.0-1)/5 = 0.20

Streamlit Implementation

```
questions = {'AW': [...], 'DB': [...], 'SV': [...]}
scores = {}
for dim, items in questions.items():
    vals = [st.slider(q, 1, 6, 3) for q in items]
    scores[dim] = sum(vals) / len(vals)
    risks[dim] = (scores[dim] - 1) / 5
```

4. Phishing Simulation Design

6 messages per test (2 per dimension), randomly selected from a bank of 18 messages (6 per dimension). Simulation 2 uses different versions than Simulation 1.

Dim	Tactic	Purpose	Example Red Flag
AW	Institutional impersonation	Check if student notices fake sender/domain	Fake: jce-update.com Real: jce.ac.il
AW	Legitimate institutional message	Check if student avoids over-suspicion	Correct domain, no urgency, no suspicious link
DB	Quiet phishing	Check behavior with no emotional pressure	Routine library message with bad link
DB	Data-entry request / suspicious link	Check if student trusts a link asking for data	Form requesting sensitive details via email link
SV	Urgency / threat	Check if time pressure affects the decision	"Account suspended in 24 hours"
SV	Financial reward / social engineering	Check if temptation or authority affects decision	"You won a 5000 scholarship" or "Urgent request from Dean"

Message JSON Schema (messages.json)

```
{ "id": "AW_01", "dimension": "AW", "tactic": "Institutional impersonation", "from_name": "JCE Security Team", "from_email": "security@jce-update.com", "subject": "Urgent security update for your account", "body": "Dear student, unusual activity detected...", "correct": "Phishing", "why": "Domain jce-update.com is not the real jce.ac.il", "teach": "Always compare the domain after @ with the official one", "red_flags": ["fake domain", "urgency", "generic greeting"], "difficulty": "easy" }
```

Email Count

Dimension	Sim 1	Sim 2	Reserve	Total
AW	2	2	2	6
DB	2	2	2	6
SV	2	2	2	6
TOTAL	6	6	6	18

Email Display

Each email is rendered as an HTML card in Streamlit (st.markdown with unsafe_allow_html=True). The card shows: sender name, sender email (domain visible), subject, body, and a fake non-clickable link. More realistic than AI-generated images.

5. Hidden Timer Logic & Penalty

The timer starts when a simulation message is displayed and stops when the student submits an answer. It is **hidden from the student** so it measures natural behavior.

Timer Interpretation

Case	Interpretation
Wrong + very fast (<5s)	Rushed decision; insufficient checking
Wrong + slow (>15s)	Student tried but may lack knowledge
Correct + fast	Confidence or strong familiarity; do not penalize
Correct + slow	Careful checking behavior

Timer Penalty Formula

Condition	Penalty Added
Wrong answer in < 5 seconds	+0.20
Wrong answer in 5-15 seconds	+0.10
Wrong answer in > 15 seconds	+0.00
Correct answer (any time)	+0.00

The timer is a **supporting behavioral indicator**. It only adds penalty when the answer is wrong — it never penalizes correct answers regardless of speed.

Streamlit Implementation

```
import time

# When displaying email:
start_time = time.time()

# When student submits answer:
response_time = time.time() - start_time
st.session_state.response_times.append(response_time)
```

6. Risk & Gap Calculation

CRITICAL: Python performs ALL calculations. GPT does NOT calculate anything.

Step 1: Actual Risk

```
Error_Rate = number_of_errors_in_dimension / number_of_messages_in_dimension
Timer_Penalty_Avg = average of timer penalties for wrong answers in dimension

Actual_Risk = min(1.0, Error_Rate + Timer_Penalty_Avg)
```

Step 2: Gap Score

```
Gap = Actual_Risk - Self_Report_Risk
```

Positive gap = student is **worse** than they think. Negative gap = student is **better** than they think.

Step 3: Gap Categories

Gap Value	Gap Type	Style
+0.50 or higher	Large gap	Direct but respectful confrontation
+0.25 to +0.49	Medium gap	Gentle warning
-0.24 to +0.24	Approximately aligned	Support or targeted practice
-0.25 to -0.49	Better than expected	Confidence reinforcement
-0.50 or lower	Much better than expected	Strong confidence reinforcement

Complete Example

Dim	Survey	Self Risk	Errors	Timer Penalty	Actual Risk	Gap	Gap Type
AW	2.0	0.20	1/2	0.00	0.50	+0.30	Medium
DB	1.5	0.10	2/2	+0.15	1.00	+0.90	Large
SV	5.0	0.80	1/2	0.00	0.60	-0.20	Aligned

Python Implementation

```
def calc_actual_risk(errors, total, timer_penalties):
    error_rate = errors / total
    penalty_avg = sum(timer_penalties)/len(timer_penalties) if timer_penalties else 0
    return min(1.0, error_rate + penalty_avg)

def calc_gap(actual_risk, self_risk):
    return round(actual_risk - self_risk, 2)

def gap_type(gap):
    if gap >= 0.50: return 'large'
    if gap >= 0.25: return 'medium'
    if gap >= -0.24: return 'aligned'
    if gap >= -0.49: return 'better'
```



```
return 'much_better'
```

7. Error Type → Recommendation Content

The content of the recommendation is determined by **which specific tactic the student failed**.

If Student Failed In...	Teach This Content
Institutional impersonation	How to check official domains, sender addresses, and institutional wording. Compare real vs fake domains.
Legitimate message	How to avoid over-suspicion and recognize signs of real institutional communication.
Quiet phishing	Why links/domains must be checked even when there is no pressure or threat. Complacency is the risk.
Data-entry request	Why sensitive information should never be entered through a message link without verification.
Urgency / threat	How urgency and fear are used to push fast decisions. No legitimate email threatens account closure.
Financial / social engineering	How rewards, authority, and personal requests can manipulate decisions. Verify via official channels.

8. Gap Type → Recommendation Style

The **same error** can receive a **different tone** depending on the gap between questionnaire and simulation.

Gap Type	Meaning	Example Tone
Large gap	Student thought they were strong, but behavior was weak	"Your results show a clear gap between your self-evaluation and your actual behavior."
Medium gap	Partly aware, but performance weaker than expected	"The simulation suggests this issue affected you more than you expected."
Aligned	Self-report and performance are consistent	"Your self-evaluation was accurate; now focus on targeted practice."
Better than expected	Student underestimated their ability	"You performed better than you expected — build confidence in your process."
Much better than expected	Student strongly underestimated ability	"Your performance was strong; trust your careful checking process."

9. GPT API Role & Prompt Template

RULE: GPT receives pre-computed results and writes a clear personalized explanation. It does NOT calculate scores, risks, or gaps.

Data Sent to GPT

- Questionnaire scores for AW, DB, SV (raw averages)
- Self-report risk values (0-1 scale)
- Actual risk values (0-1 scale, including timer penalty)
- Gap values and gap types (pre-computed by Python)
- List of errors: which tactics failed, correct answers, response times
- Teaching content associated with each error (from messages.json)

System Prompt (copy exactly)

You are an educational recommendation system that helps students improve resilience against phishing.

Write in simple, clear language.

Do not invent results that are not in the data.

Use these rules:

1. Error type decides what to teach.
2. Gap type decides how to teach.
3. The timer is only a supporting behavioral indicator.
4. If the gap is large, use direct but respectful confrontation.
5. If the gap is medium, use gentle warning.
6. If aligned, use support and targeted practice.
7. If better than expected, reinforce confidence.

Write a Markdown report with:

- Personal diagnosis (2-3 sentences, address student by name)
- Why the errors happened (reference specific messages)
- Three practical recommendations
(content from error type, tone from gap type)
- One actionable tip per recommendation
- Short encouraging closing

User Message Template

Student: {name}

Questionnaire: AW={aw_score}, DB={db_score}, SV={sv_score}

Self-report risks: AW={aw_risk}, DB={db_risk}, SV={sv_risk}

Actual risks: AW={aw_actual}, DB={db_actual}, SV={sv_actual}

Gaps: AW={aw_gap} ({aw_gap_type}), DB={db_gap} ({db_gap_type}),
SV={sv_gap} ({sv_gap_type})

Errors:

- Message AW_01: {tactic}, answered {student_answer}, correct was {correct}, response time: {time}s. Teach: {teach_content}
- ... (list all errors)

Write a personalized recommendation report.

Streamlit API Call

```
import anthropic # or: import openai

client = anthropic.Anthropic()
response = client.messages.create(
    model='claude-sonnet-4-20250514',
    system=SYSTEM_PROMPT,
    messages=[{'role':'user', 'content':user_data}],
    max_tokens=1500
)
recommendation = response.content[0].text
st.markdown(recommendation)
```

10. User Flow — Single Session (~22 min)

#	Step	Duration	Details
1	Student opens system	—	No login required
2	Questionnaire (AW, DB, SV)	4 min	9 items, Likert 1-6
3	Python: calculate self-report risks	instant	(score-1)/5 per dimension
4	Select 6 simulation messages	instant	2 per dimension, random from bank
5	Show message + hidden timer starts	—	Per message
6	Student answers Phishing/Legitimate	~50s each	Timer stops, response saved
7	Python: correct answers, group errors	instant	By dimension
8	Python: calculate actual risks + gaps	instant	Error_Rate + Timer_Penalty
9	Python: determine gap types	instant	5 categories by threshold
10	Python: map error → content, gap → style	instant	Lookup tables
11	GPT: generate recommendation	~3 sec	Receives all pre-computed data
12	Display recommendation to student	5 min	Student reads Markdown report
13	Simulation 2 (6 NEW messages)	5 min	Different versions, same tactics
14	Python: compare Sim1 vs Sim2	instant	Improvement delta
15	Satisfaction survey (5 questions)	2 min	1-5 scale
16	Display final report + save CSV	1 min	Before/after + satisfaction

Satisfaction Survey (5 questions, 1-5 scale)

#	Question
1	Were the recommendations clear and understandable?
2	Did you feel the recommendations were personalized to you?
3	Did you learn something new that you didn't know before?
4	Would you recommend this system to your classmates?
5	Did the system help you improve your digital behavior?

11. Data Storage Schema (CSV)

Save every completed test to a CSV file. This provides data for the final graduation report.

Field	Meaning
student_id	Anonymous participant ID
timestamp	Date and time of the test
AW_score, DB_score, SV_score	Questionnaire averages (1-6)
AW_self_risk, DB_self_risk, SV_self_risk	Self-report risk (0-1)
AW_actual, DB_actual, SV_actual	Simulation-based actual risk (0-1)
AW_gap, DB_gap, SV_gap	Gap values (actual - self)
AW_gap_type, DB_gap_type, SV_gap_type	large/medium/aligned/better/much_better
sim1_score	Simulation 1 correct count (/6)
sim2_score	Simulation 2 correct count (/6)
improvement	sim2_score - sim1_score
errors_json	JSON list of wrong answers + tactics
response_times_json	Response time for every message (seconds)
gpt_recommendation	Full recommendation text
satisfaction_avg	Average satisfaction score (1-5)

CSV Implementation

```
import csv, json, os

def save_result(data):
    file = 'results.csv'
    exists = os.path.exists(file)
    with open(file, 'a', newline='') as f:
        w = csv.DictWriter(f, fieldnames=data.keys())
        if not exists: w.writeheader()
        w.writerow(data)
```

12. File Structure & Deployment

```
phishing-system/
|-- app.py           # Main Streamlit app (~250 lines)
|-- scoring.py       # Risk, gap, timer calculations
|-- prompts.py       # System prompt + user template
|-- questionnaire.py # 9 questions + scoring
|-- messages.json    # 18 email templates
|-- results.csv      # Auto-generated data file
|-- requirements.txt  # streamlit, anthropic
```

```
| -- .streamlit/  
|   |-- secrets.toml    # ANTHROPIC_API_KEY=...  
|-- README.md
```

Deployment (Streamlit Cloud — Free)

- Push code to GitHub
- Go to share.streamlit.io, connect repo
- Add API key in Secrets management
- Share URL with students
- Total cost: \$0 (free tier + API pay-per-use)

13. Success Metrics

Metric	Target	How to Compute
SimScore Improvement	mean(Sim2) >= 4.20/6 (midterm avg: 3.62)	mean(sim2_score) across all participants
User Satisfaction	>= 4.0 / 5	mean(satisfaction_avg) across all participants
Data Completeness	100%	completed / started (should be 100% in single session)
Gap Detection	Qualitative validation	Review AI outputs for students with known large gaps

14. Testing Scenarios

Scenario	Expected Behavior
Confident student, many mistakes	Large gaps in AW/DB; direct confrontation style; recommendation references specific failed messages
Confident student, one mistake	Medium gap; gentle warning; focused on the single weak tactic
Student admits weakness, performs weakly	Aligned gaps; supportive tone; targeted practice recommendations
Student reports weakness, performs well	Better-than-expected; confidence reinforcement; "trust your process" messaging
Student answers everything correctly	Positive feedback; maintenance tips; no confrontation needed

15. Defense Explanation

Use this explanation in your presentation or final report:

The system combines a self-report questionnaire, a phishing simulation with hidden response-time measurement, and AI-generated personalized recommendations. Python calculates the gap between each student's self-evaluation and their actual performance across three dimensions (Awareness, Behavior, Vulnerability). The AI then generates a personalized recommendation where the error type determines the content and the gap type determines the tone. A second simulation immediately measures improvement. This design is grounded in our midterm finding that self-report explains only 2.5% of performance variance — proving that questionnaires alone are insufficient.

16. Future Work

Feature	Description	Complexity
Multi-session paths	3 tracks (single/week/3-week) with spaced repetition	Medium — needs DB
Dynamic email generation	AI generates new emails per student (infinite variety)	Low — extra API call
Adaptive difficulty	Increase email difficulty based on performance	Medium
Progress dashboard	Visual chart of improvement across sessions	Medium — needs DB
Behavioral questions	"What did you check first?" after each email	Low — optional
Smart welcome screen	Shows progress when student returns	Low — needs DB

Appendix A: Conflict Resolution Log

This document was merged from two versions. Below are all conflicts and how each was resolved:

Conflict	V1 (Reference)	V2 (Claude)	Resolution
Questionnaire items	9 items (3 per dim)	20 items	9 items — simpler for MVP
Who computes gaps	Python	AI (80%)	Python — deterministic & reproducible
Time measurement	Hidden timer + penalty formula	Behavioral question asked to student	Hidden timer — more accurate
Post-message questions	Removed from MVP	2 questions per email	Removed — timer replaces them
Gap categories	5 numeric thresholds	3x3 descriptive matrix	Numeric thresholds — easier to code
Data storage	CSV file	session_state only	CSV — need data for final report
GPT output format	Markdown	JSON	Markdown — simpler, no parsing needed

End of Document — This is the single source of truth for the system. Any AI tool can use this document to understand the complete architecture, formulas, data flow, and implementation requirements.